

Challenge Clyvo — Olli Pet

Apresentação do Trabalho

DevOps Tools & Cloud Computing — Sprint 3

Time Olli Pet — Andre Rosa Colombo (RM563112), Gabriely Bonfim Silva (RM566242), Henrique Rodrigues Vespasiano (RM562917), Mirelly Sousa Alves (RM566299) e Ruan Luca Feliciano (RM562218) — Turma 2TDSA.

Este documento apresenta o que foi entregue na Sprint 3 da disciplina de DevOps Tools & Cloud Computing: a escolha de arquitetura, como cada requisito do enunciado foi atendido, e as evidências (prints e saídas de comando reais) de que a aplicação está de pé, tanto local quanto na Azure. Não substitui o PDF oficial de entrega, que contém só nomes, RMs e links.

Repositório GitHub	github.com/HenriqueRodriguesV/java-ollipet-devops
API em produção (Azure)	ollipet-rm562917.eastus.azurecontainer.io:8080
Opção escolhida	ACR + ACI (containerização completa)
Banco de dados	PostgreSQL 16, em container
Vídeo de apresentação	(cole aqui o link do YouTube após gravar)

1. Sobre o projeto

A Olli Pet é uma clínica veterinária fictícia (base do Challenge Clyvo) que hoje resolveria agendamento, carteira de vacinação, triagem de urgência e acompanhamento de tratamento por WhatsApp e planilha. Construímos uma API em Spring Boot que é o backend do aplicativo mobile da clínica e concentra quatro regras de negócio:

- **Triagem por questionário** — classifica a urgência de um pet a partir das respostas do tutor.
- **Agendamento de consulta** — controla a agenda dos veterinários, evita choque de horário.
- **Carteira de vacinação** — calcula sozinha o número da dose e a próxima aplicação.
- **Tratamento em casa** — transforma uma prescrição em doses individuais confirmáveis pelo tutor.

Por que isso importa pro negócio

Hoje a agenda depende de alguém da recepção lembrar de conferir choque de horário ou cancelamento fora do prazo; a API recusa isso sozinha. A triagem também deixa de depender de alguém da clínica estar disponível no WhatsApp — o tutor recebe uma classificação de urgência padronizada a qualquer hora. No lado da vacinação, o cálculo automático da próxima dose evita descobrir vacina atrasada só quando o tutor aparece. E no tratamento em casa, o veterinário passa a ver um percentual real de doses confirmadas, em vez de depender do relato do tutor.

2. O que a Sprint 3 pedia

O enunciado de DevOps Tools & Cloud Computing pedia para escolher entre duas opções de entrega, sem misturar as duas: **ACR + ACI** (containerização completa de aplicação e banco) ou **App Service + banco PaaS** (nada containerizado). Optamos pela primeira, porque já tínhamos a aplicação dockerizada desde a sprint anterior e o material de aula (Aula 32) trabalhava justamente com Azure Container Registry e Azure Container Instances.

3. Como cada requisito foi atendido

Requisitos gerais (independem da opção escolhida):

- ✓ Descrição da solução e dos benefícios — seções 1 e 2 acima, e no README do repositório.
- ✓ Banco de dados em nuvem, não é H2 — PostgreSQL 16 rodando em container.
- ✓ DDL das tabelas, com comentários, em arquivo separado — `script_bd.sql`, na raiz do repositório.
- ✓ Tabelas do CORE da aplicação (não cadastro genérico) — `pet`, `responsavel`, `med_vet`, `consulta`, `vacina`, `tratamento` etc.
- ✓ CRUD completo em pelo menos duas tabelas relacionadas — `pet` e `responsavel` (chave estrangeira `pet.id_resp`), evidenciado na seção 6.
- ✓ Pelo menos duas linhas manipuladas com conteúdo significativo — a carga inicial cadastra 3 pets (Thor, Nina, Bud); testamos criando, atualizando e removendo registros adicionais.
- ✓ Código-fonte publicado no GitHub — repositório público, histórico de commits real.

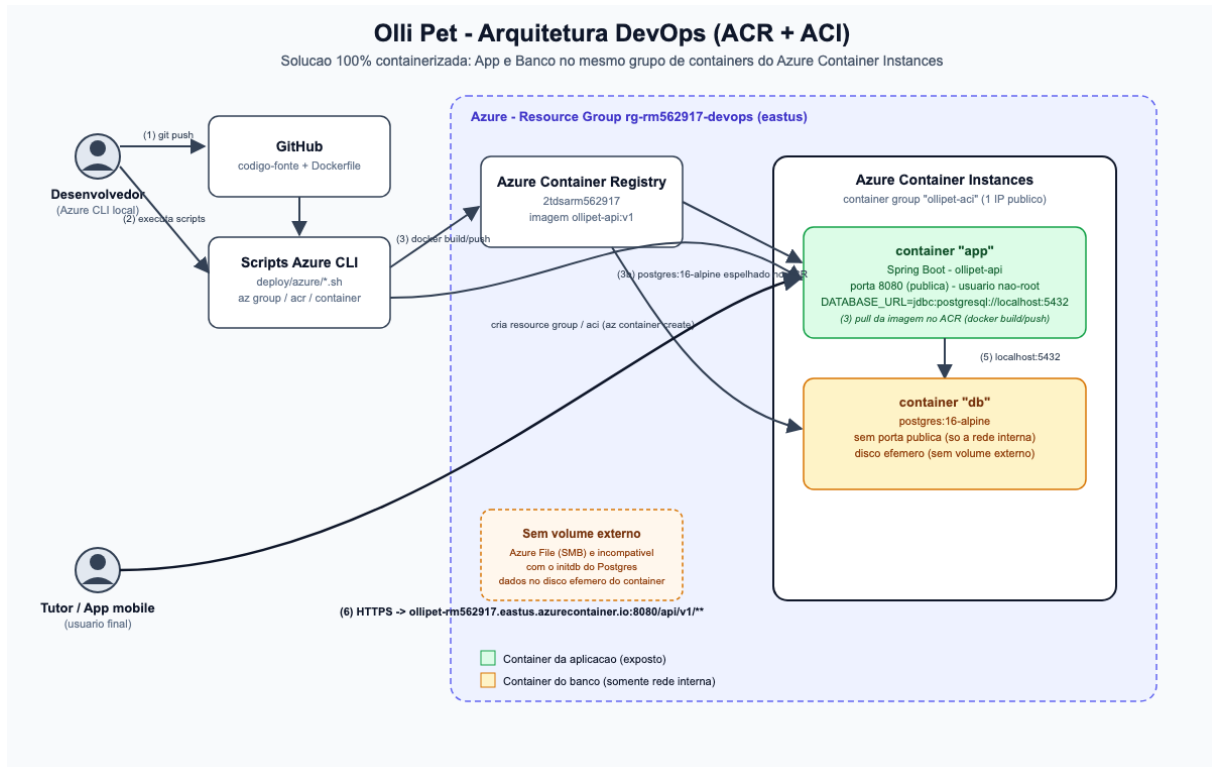
Requisitos específicos da opção ACR + ACI:

- ✓ Todos os recursos (Resource Group, ACR, ACI) criados via Azure CLI — nenhum recurso foi criado manualmente pelo Portal.
- ✓ Container da aplicação não roda como `root/admin` — usuário dedicado criado no Dockerfile.
- ✓ Dockerfile e `docker-compose.yml` entregues no repositório.
- ✓ Scripts de build e execução da imagem entregues — pasta `deploy/azure/`, com um script por etapa.

As duas seções seguintes mostram exatamente isso funcionando: a arquitetura da solução e depois a aplicação rodando, com prints e saídas reais de comando.

4. Arquitetura da solução

Um único container group do Azure Container Instances roda os dois containers lado a lado, compartilhando a mesma rede interna. Isso significa que o Postgres nunca precisa ficar exposto na internet — só a porta 8080 da API é pública.



GitHub → scripts Azure CLI → build/push da imagem no ACR → container group no ACI → API pública.

5. Rodando localmente

Antes de mexer em qualquer coisa na nuvem, validamos tudo localmente com Docker Compose — sobe a API e o Postgres juntos, e o Flyway aplica as migrations automaticamente no boot:

```
$ docker compose up --build

Container ollipet-db   Started
Container ollipet-api  Started
```

Swagger em <http://localhost:8090/swagger-ui.html>, com todos os endpoints da API:

The screenshot displays the Swagger UI interface for an API. At the top, the Swagger logo is visible, along with a search bar containing "/api-docs" and an "Explore" button. Below the header, the text "OpenAPI definition" is shown, followed by "v0" and "OAS 3.1". A "Servers" section indicates the server URL as "http://localhost:8090 - Generated server url". The main content area is divided into two sections: "Prontuarios" (Historico clinico dos pets) and "Pets" (Cadastro de pets). The "Prontuarios" section lists three endpoints: a GET request for "/api/v1/prontuarios/{id}" (Buscar prontuario por ID), a GET request for "/api/v1/prontuarios/pet/{petId}" (Historico clinico de um pet), and a GET request for "/api/v1/prontuarios/meus" (Prontuarios que eu registrei). The "Pets" section lists five endpoints: a DELETE request for "/api/v1/pets/{id}" (Remover pet), a GET request for "/api/v1/pets/{id}" (Buscar pet por ID), a GET request for "/api/v1/pets" (Listar pets da clinica), a GET request for "/api/v1/pets/meus" (Meus pets), and a POST request for "/api/v1/pets" (Cadastrar pet).

6. CRUD completo — validado com SELECT direto no banco

Testamos Create, Read, Update e Delete no recurso **Pet**, que é relacionado à tabela **Responsavel** pela chave estrangeira `pet.id_resp`. Depois de cada chamada, conferimos com um `SELECT` direto no PostgreSQL — não é dado mockado em memória, é o banco de verdade.

CREATE — POST `/api/v1/pets`

```
{
  "id": 4,
  "nome": "Bolinha",
  "raca": "Vira-lata",
  "especie": "CAO",
  "dataNascimento": "2024-02-10",
  "responsavelId": 3,
  "nomeResponsavel": "Maria Silva"
}
```

UPDATE — PUT `/api/v1/pets/4`

```
{
  "id": 4,
  "nome": "Bolinha Castrado",
  "raca": "Vira-lata",
  "especie": "CAO",
  "dataNascimento": "2024-02-10"
}
```

Conferindo direto no Postgres (confirma que INSERT e UPDATE persistiram de verdade)

```
id_pet | nome           | raca           | id_resp
-----+-----+-----+-----
1 | Thor           | Golden Retriever | 3
2 | Nina           | Siames          | 3
3 | Bud            | Poodle          | 4
4 | Bolinha Castrado | Vira-lata       | 3
(4 rows)
```

DELETE — DELETE `/api/v1/pets/4` → HTTP 204, e o SELECT confirma a remoção

```
id_pet | nome | raca           | id_resp
-----+-----+-----+-----
1 | Thor | Golden Retriever | 3
2 | Nina | Siames          | 3
3 | Bud  | Poodle          | 4
(3 rows) -- linha 4 sumiu, os dados originais continuam intactos
```

7. Deploy na Azure

Com tudo validado localmente, criamos os recursos na Azure via CLI, na ordem descrita no README do repositório (resource group, ACR, build e push da imagem, container group). Este é o estado real dos recursos depois do deploy:

```
$ az resource list --resource-group rg-rm562917-devops -o table
```

Name	Location	Type	Status
2tdsarm562917	eastus	ContainerRegistry/registries	Succeeded
ollipet-aci	eastus	ContainerInstance/containerGroups	Succeeded

```
{
  "status": "Running",
  "fqdn": "ollipet-rm562917.eastus.azurecontainer.io",
  "containers": [
    { "nome": "db", "estado": "Running", "restarts": 0 },
    { "nome": "app", "estado": "Running", "restarts": 1 }
  ]
}
```

E a mesma API, agora publicada (repare a URL do servidor no Swagger):

Swagger
Supported by SMARTBEAR

/api-docs

Explore

OpenAPI definition v0 OAS 3.1

Servers

http://ollipet-rm562917.eastus.azurecontainer.io:8080 - Generated server url

Prontuarios Historico clinico dos pets

- GET /api/v1/prontuarios/{id} Buscar prontuario por ID
- GET /api/v1/prontuarios/pet/{petId} Historico clinico de um pet
- GET /api/v1/prontuarios/meus Prontuarios que eu registrei

Pets Cadastro de pets

- DELETE /api/v1/pets/{id} Remover pet
- GET /api/v1/pets/{id} Buscar pet por ID
- GET /api/v1/pets Listar pets da clinica
- GET /api/v1/pets/meus Meus pets
- POST /api/v1/pets Cadastrar pet

8. CRUD completo — validado direto na API pública

Repetimos o mesmo teste do item 6, agora contra o ambiente publicado, não localhost:

```
CREATE -> HTTP 200 { "id": 5, "nome": "Nuvem", "raca": "SRD", ... }  
READ   -> HTTP 200 { "id": 5, "nome": "Nuvem", ... }  
UPDATE -> HTTP 200 { "id": 5, "nome": "Nuvem Atualizado", ... }  
DELETE -> HTTP 204  
READ (apos DELETE) -> HTTP 404 (confirma remocao)
```

No vídeo, o SELECT direto no banco em nuvem é feito abrindo um shell interativo dentro do container do Postgres (o roteiro completo está em docs/roteiro-video-sprint3.md).

9. Código no GitHub

Repositório público, histórico de commits real (não é um upload único):

HenriqueRodriguesV / java-ollipet-devopsPublic

Notifications

<> Code

Issues

Pull requests

Actions

Projects

Security and

main

Go to file

<> Code

HenriqueRodriguesV and Reescreve secao de beneficios d...a54d198 · now

claude

.mvn/wrapper	Base do projeto Olli Pet (Sprin...	15 hours ago
deploy/azure	Ajusta valores de exemplo nos...	16 minutes ago
docs	Adiciona relatorio tecnico com...	7 minutes ago
src	Base do projeto Olli Pet (Sprin...	15 hours ago
.dockerignore	Base do projeto Olli Pet (Sprin...	15 hours ago
.env.example	Ajusta valores de exemplo nos...	16 minutes ago
.gitattributes	Base do projeto Olli Pet (Sprin...	15 hours ago
.gitignore	Base do projeto Olli Pet (Sprin...	15 hours ago
Dockerfile	Containeriza a aplicacao para ...	15 hours ago
README.md	Reescreve secao de beneficio...	now
docker-compose.yml	Containeriza a aplicacao para ...	15 hours ago
mvnw	Base do projeto Olli Pet (Sprin...	15 hours ago
mvnw.cmd	Base do projeto Olli Pet (Sprin...	15 hours ago
pom.xml	Base do projeto Olli Pet (Sprin...	15 hours ago
script_bd.sql	Adiciona scripts de deploy na ...	15 hours ago

README

Challenge Clyvo - Olli Pet (API)

API REST em Spring Boot que atende o aplicativo mobile da clinica veterinaria Olli Pet. Toda a superficie publica esta em `/api/v1/**`.

Vídeo de apresentação: ([disponível no youtube](#))

Vídeo de apresentação: ([disponível no Google Drive](#))

Repositório de Java: ([repositório](#))

Repositório de Mobile (frontend): ([repositório](#))

Descrição da solução

github.com/HenriqueRodriguesV/java-ollipet-devops

10. Decisões e dificuldades no caminho

Vale registrar o que não saiu de primeira, porque foi onde mais aprendemos:

- **Azure File Share para persistir o Postgres:** tentamos montar um volume de arquivo da Azure pro diretório de dados do Postgres, mas o `initdb` trava nesse tipo de armazenamento de rede (Postgres depende de locking/fsync que o SMB não garante direito). Decidimos deixar o banco no disco do próprio container: atende o requisito de "banco em container", só não sobrevive se o container do banco for recriado do zero.
- **Imagem ARM64 x AMD64:** como o build local roda num Mac com chip Apple Silicon, a imagem Docker saía em ARM64 por padrão — e o ACI só aceita AMD64. Resolvido forçando `--platform linux/amd64` no build.
- **az acr build indisponível:** a subscription Azure for Students não permite ACR Tasks (build remoto direto no registry). Voltamos pro fluxo clássico: build local, docker login, docker push.
- **Rate limit do Docker Hub a partir do ACI:** o pull da imagem oficial do Postgres direto do Docker Hub estava sendo recusado às vezes a partir do datacenter da Azure. Resolvido espelhando a imagem no nosso próprio ACR.

11. Vídeo de demonstração

([cole aqui o link do YouTube apos gravar](#))